The one-dispatch fold: a class-partitioned execution plan for the BCH DAG

lie-rs / signature-rs — design note, 2026-09-01. For readers who know the code **before** the class-contiguous work: `LieSeries`, the batch kernel (`commutator_coefficients_batch_with_cache`: gating prologue → (job, unit) bundles → parallel sweep), and the DAG fold's level-parallel `evaluate`. Everything newer is defined here.

The fold today: one dispatch per dependency level

A fold evaluates the DAG of bracket nodes level by level: level **l**'s nodes are commutator kernel calls whose operands are results of levels < **l**. Today each level runs as its own rayon batch — prologue, task flattening, dispatch, join — and the calling thread coordinates every one of them. A fold with **L** levels pays **L** dispatch/join round-trips, and for small grids those round-trips are a large share of the fold (≈35% of a 3×8 fold is rayon bookkeeping, not sweeping).

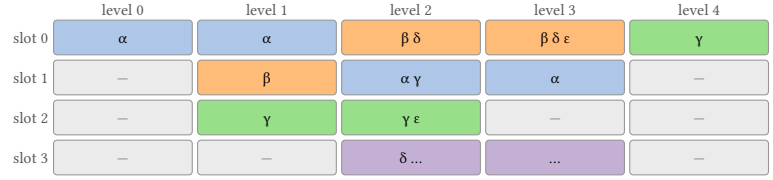
Today: the calling thread runs one rayon dispatch + join per level. Dark = coordination that produces no coefficients.

Two further costs hide inside the sweeps. In the class-contiguous layout each node's result is written to a scratch buffer and permuted back per call, and each level re-derives its gating from scratch. Both are per-level costs that a fold-level schedule can amortize.

The design: plan the whole fold, dispatch it once

The observation is that a fold is **just** a sequence of independent work sets with a total order between them: level **l**'s jobs have disjoint result buffers, and read only outputs of levels < **l**. So the entire fold can be handed to the pool as one parallel operation, with the dependency order enforced by cheap per-level counters instead of dispatch boundaries. Three steps:

1. Plan everything up front (serial, before any sweeping). The support lists of every node are fixed for the whole fold (they are the interned DAG's scatter-target fixed point), so every job's gating — presence bitsets and active-unit segments — can be computed for **all** levels in one pass over the shared gating cache. Each level's (job, unit) tasks are then cut into **packs**: balanced ranges, always cut at unit boundaries, sized so there is one pack per available worker. Because each DAG node is exactly one anagram class (a bracket `[X, Y]` only activates units with γ = γ_X + γ_Y), and a level's tasks are node-ordered, every pack is class-contiguous **by construction** — one pack sweeps a few whole classes back-to-back.



The pack-slot walk (illustrative: 4 slots, 5 levels, classes α–ε as colors). One rayon dispatch total: each slot claims packs off every level's cursor, top to bottom. Between levels, a slot waits on a counter until **all** packs of the level are done — that counter is what orders cross-level operand reads.

2. Execute as one parallel walk. The engine spawns exactly one task per pack slot — never more than the worker count — and each task walks the levels: sweep my pack of level **l**; bump level **l**'s counter (`Release`); spin until the counter reaches the level's pack count (`Acquire`); continue. The `Release`/`Acquire` pair is the only synchronization: it publishes a level's buffer writes to the readers of the next level, replacing **L** dispatch/joins with **L** counter bumps inside a single dispatch.

3. Keep the accumulation contract. Packs are cut at unit boundaries and never split a unit, and each result word belongs to exactly one unit — so every word is still accumulated by its own entry sequence, in order, regardless of which slot runs it. Results are bit-identical to the per-level fold; the oracle and round-trip tests pass with the walk enabled.

The sequence of work, slot by slot

Each slot task is the same small loop; the per-level counters are its only shared state. In words: **wait at the level boundary, sweep only if I hold a pack, report only if I swept.**

Worked example — 3 slots, 3 levels, pack counts (1, 3, 2); only slot 0 has a level-0 pack:

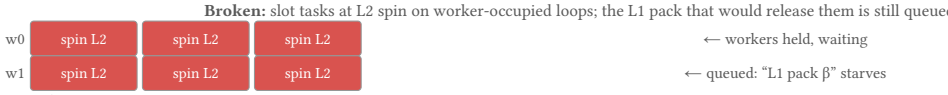
phase	slot 0	slot 1	slot 2
sweep L0	sweeps its pack	reports immediately — no pack	reports immediately
gate L1	proceeds	proceeds	proceeds
sweep L1	— no pack	sweeps its pack	sweeps its pack
gate L2	proceeds when $done_1 \geq 3$	proceeds when $done_1 \geq 3$	proceeds when $done_1 \geq 3$
sweep L2	sweeps its pack	— no pack	sweeps its pack
counters	$done_0 = 1$ ✓ · $done_1 = 3$ ✓ · $done_2 = 2$ ✓ — fold complete		

Two rules make the sequence watertight. **Every slot waits at every level boundary** — including slots that did nothing at the previous level — so no pack of level **l** can read a buffer of level **l**−1 before all packs of level **l**−1 have released their work. And **only a slot holding a pack reports the level**: if empty-pack slots reported for free, a level could appear complete before its packs have run, and level **l**+1 would read buffers level **l** is still writing. Reports are the only shared writes; the sweeps themselves need no atomics because their packs write disjoint words. The gate phase costs one spin-now-yield-later loop per slot per level — a counter test, where a dispatch used to be.

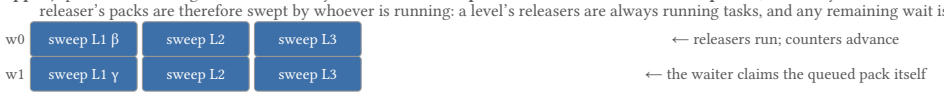
The blocker: waiters must not hold workers

The counter wait is the one dangerous part. If a waiting slot **occupies its worker** (spin, or worse, sleep) while the pack that would release it still sits in rayon's queue, the queued pack can starve: under heavy oversubscription (the full test suite runs many processes × 32 workers), every worker can end up held by a waiter, and the fold never completes.

Broken: slot tasks at L2 spin on worker-occupied loops; the L1 pack that would release them is still queued.



Fix (shipped): packs are not assigned to slots — they are **claimed off a per-level atomic cursor at sweep time**, and every waiter drains that cursor before it blocks. A queued releaser's packs are therefore swept by whoever is running: a level's releasers are always running tasks, and any remaining wait is bounded by real sweeps.



Status and payoff

Shipped as the default (`commutator_coefficients_class_fold_with_cache`) and bit-identical: packs are claimed dynamically, every waiter drains the claim cursor before blocking, and a nested-parallelism stress test (`dag_fold_survives_nested_parallel_oversubscription`) guards the liveness. A single-worker pool routes to the per-level serial sweep — identical per-word order, no fold planning. Measured, 32 workers: 3×8 fold 0.54ms vs 1.23ms per-level (2.3×); 2×12 fold 2.4ms vs 3.7ms (1.5×). Payoff: it removes the per-level dispatches — ≈35% of a small fold's time — and is the prerequisite for the next step, **pipelining a batch of folds through the same pack slots**, where fold **n**'s narrow first levels fill the workers while fold **n**−1 sweeps its wide middle. Since the pack plan depends only on the basis, a batch of folds over one series reuses both the ordering and the packs for free: the per-fold planning cost stays O(1).

Pipelining (shipped): one dispatch per batch of folds

Shipped as `concatenate_batch_coefficients` (drives `build_from_path`): after the first few folds warm the DAG to its steady state, **all** remaining folds run as ONE continuous walk — Z-stage zeroing, per-fold gather and accumulate as parallel block stages between sweep levels, the class-space accumulator persisting across folds (fold **n**'s operand **is** fold **n**−1's result — no gather/epilogue per fold), one plan and one job table for the whole batch. Eligibility is checked once per batch: shared displacement support, node lists at their fixed point, and the accumulator's support equal to the reachable set — the level-0 masks then cover every position later folds can touch, so mid-batch cancellations stay sound. Bit-identical to per-fold concatenation (float order preserved; exact-rationals test green), incl. per-job operand masks copied from the per-fold path: interior jobs gate on their children's scatter lists, not the atom supports.

Measured (min-of-5, per fold; kernel bench, dense jobs, 64 folds/install): 3×8 — 20.9⟶19.9µs at 1t, 23.9⟶8.7µs at 8t (2.8×), 49.8⟶9.3µs at 32t (5.3×); 8×3 — 14.0⟶2.9µs at 8t (4.8×), 43.7⟶5.9µs at 32t (7.4×); 2×12 44.1⟶5.5µs at 32t (8.0×). End-to-end `build_from_path` 3×8×1001: 533.9⟶468.9ms at 32t (−12%; the residual is the shared per-fold gating prologue, not the sweeps). Targets met: 3×8 batch ≤0.80ms (actual 0.597ms), 4×10 ≤154ms (actual 32.5ms). Remaining lever: the prologue/accumulate share — the next fixes (B: parallel accumulate; C: pool right-sizing) target exactly that.